Міністерство освіти і науки України
Львівський національний університет імені Івана Франка
Факультет електроніки та комп'ютерних технологій
Кафедра радіофізики та комп'ютерних технологій

Звіт

З лабораторної №9

З дисципліни «Професійний Python»

На тему **«Паралельність, багатопроцесність та оптимізація»**

Виконав:
Студент 3 курсу
групи ФеП-31
Войтович М.А.
Перевірив:
проф. Свелеба С. А.

Львів 2026

Варіант: 4 (Система банківських рахунків)

Мета: Ознайомитися з основними принципами паралельного та конкурентного виконання програм у Python, дослідити вплив Global Interpreter Lock (GIL) на CPU-bound та I/O-bound задачі. Навчитися використовувати threading, multiprocessing, ThreadPoolExecutor і ProcessPoolExecutor, проводити профілювання програм за допомогою cProfile та tracemalloc, вимірювати час виконання операцій, застосовувати NumPy для векторизації обчислень і кешування для оптимізації повторних операцій. Експериментально визначити найефективніші способи обробки великих обсягів банківських даних.

Теоретичні відомості

1. Конкурентність та паралелізм

Конкурентність (Concurrency) — це організація виконання декількох задач, за якої програма може перемикатися між ними. Паралелізм (Parallelism) передбачає одночасне виконання обчислень на декількох ядрах процесора.

2. Global Interpreter Lock (GIL)

GIL — це механізм синхронізації у стандартному CPython, який зазвичай дозволяє лише одному потоку виконувати Python-байткод в один момент часу в межах одного процесу. Через це багатопотоковість не завжди прискорює складні обчислення.

3. CPU-bound та I/O-bound задачі

CPU-bound задачі переважно навантажують процесор, наприклад обчислення статистики банківських транзакцій. I/O-bound задачі пов'язані з очікуванням операцій введення-виведення, зокрема читання файлів та звернення до бази даних.

4. Threading та multiprocessing

Threading дозволяє виконувати декілька потоків у межах одного процесу зі спільною пам'яттю. Multiprocessing використовує окремі процеси, що дає змогу задіяти декілька процесорних ядер.

5. ThreadPoolExecutor та ProcessPoolExecutor

ThreadPoolExecutor забезпечує зручне керування пулом потоків, а ProcessPoolExecutor — пулом окремих процесів для виконання незалежних задач.

6. Синхронізація та Lock

Race condition виникає, коли результат операцій залежить від порядку виконання потоків, які змінюють спільні дані. Lock забезпечує взаємне виключення під час доступу до критичної секції.

7. Профілювання за допомогою cProfile та tracemalloc

cProfile використовується для визначення функцій, які потребують найбільше процесорного часу. tracemalloc дозволяє аналізувати виділення пам'яті Python-програмою та визначати її пікове використання.

8. Векторизація NumPy

NumPy дозволяє обробляти великі числові масиви за допомогою оптимізованих операцій, зменшуючи потребу у звичайних Python-циклах.

9. Кешування (Caching)

Кешування передбачає збереження раніше обчислених результатів для прискорення повторних звернень. Після зміни вихідних даних кеш необхідно оновлювати або очищати (Cache Invalidation).

10. Benchmark та Speedup

Benchmark — це вимірювання продуктивності програмної реалізації. Для порівняння алгоритмів використовується коефіцієнт прискорення:

Speedup = T_baseline / T_optimized,

де T_baseline — час початкової реалізації, T_optimized — час оптимізованої реалізації.

Постановка задачі

Метою лабораторної роботи №9 є продовження розробки системи банківських рахунків (Варіант 4) шляхом дослідження продуктивності операцій над великими обсягами даних, реалізації паралельних обчислень та застосування методів оптимізації Python-програм.

У ході виконання роботи необхідно:

1. Зберегти основні функціональні можливості банківської системи.
2. Згенерувати тестовий набір із 100 000 банківських рахунків та 500 000 транзакцій.
3. Реалізувати послідовну Python-обробку банківських даних.
4. Дослідити продуктивність за допомогою timeit, perf_counter, cProfile та tracemalloc.
5. Реалізувати паралельну обробку з використанням ThreadPoolExecutor та ProcessPoolExecutor.
6. Дослідити вплив кількості потоків і процесів на час виконання обчислень.
7. Реалізувати векторизовані числові обчислення за допомогою NumPy.
8. Продемонструвати race condition та синхронізацію за допомогою Lock.
9. Реалізувати кешування повторних обчислень та перевірити його очищення після зміни даних.
10. Провести багаторазові вимірювання, визначити коефіцієнти прискорення та побудувати графіки.
11. Перевірити правильність реалізованих алгоритмів за допомогою pytest.

Хід виконання

1. Структура проєкту

Для виконання лабораторної роботи №9 було використано існуючий Python-проєкт bank_system, розроблений у попередніх лабораторних роботах.

Проєкт було розширено модулями для аналізу продуктивності, багатопотокового та багатопроцесного виконання операцій, кешування й автоматизованого тестування.

Основні нові компоненти:

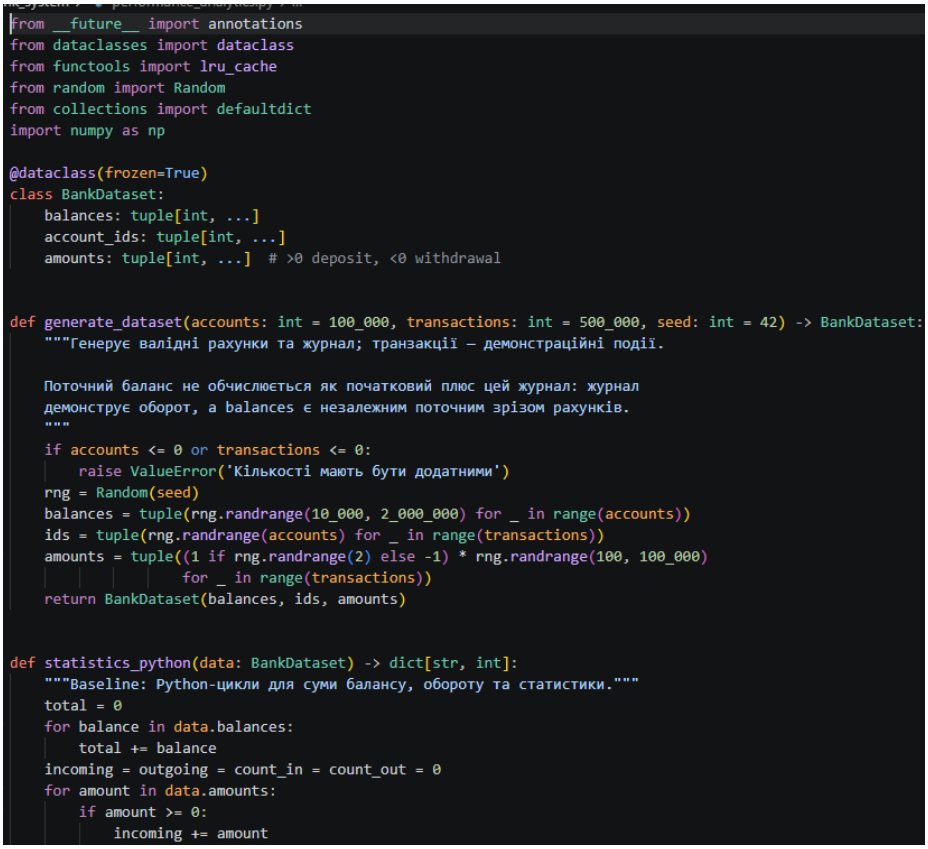
- performance_analytics.py — генерація тестових банківських даних, послідовна аналітика, векторизація NumPy та кешування.
- parallel.py — реалізація паралельної обробки за допомогою threading, multiprocessing, ThreadPoolExecutor і ProcessPoolExecutor, а також синхронізації через Lock.
- benchmark_lab9.py — автоматичне вимірювання часу виконання, порівняння реалізацій та збереження результатів.
- test_performance_lab9.py — автоматизована перевірка правильності реалізованих алгоритмів.
- benchmarks/results/ — результати профілювання, таблиці та графіки продуктивності.

2. Генерація тестових даних та послідовна реалізація

У файлі src/bank_system/performance_analytics.py реалізовано функцію generate_dataset(), яка створює тестовий набір із 100 000 банківських рахунків і 500 000 транзакцій.

Для відтворюваності результатів використовується фіксоване значення генератора випадкових чисел seed=42. Грошові суми зберігаються у цілих одиницях (копійках), що дозволяє уникнути похибок обчислення з числами з плаваючою комою.

Функція statistics_python() забезпечує послідовний підрахунок основних фінансових показників банківської системи. Ця реалізація використовується як Baseline для подальшого порівняння з оптимізованими алгоритмами.



```python
from __future__ import annotations
from dataclasses import dataclass
from functools import lru_cache
from random import Random
from collections import defaultdict
import numpy as np

@dataclass(frozen=True)
class BankDataset:
    balances: tuple[int, ...]
    account_ids: tuple[int, ...]
    amounts: tuple[int, ...]  # >0 deposit, <0 withdrawal


def generate_dataset(accounts: int = 100_000, transactions: int = 500_000, seed: int = 42) -> BankDataset:
    """Генерує валідні рахунки та журнал; транзакції — демонстраційні події.

    Поточний баланс не обчислюється як початковий плюс цей журнал: журнал
    демонструє оборот, а balances є незалежним поточним зрізом рахунків.
    """
    if accounts <= 0 or transactions <= 0:
        raise ValueError('Кількості мають бути додатними')
    rng = Random(seed)
    balances = tuple(rng.randrange(10_000, 2_000_000) for _ in range(accounts))
    ids = tuple(rng.randrange(accounts) for _ in range(transactions))
    amounts = tuple((1 if rng.randrange(2) else -1) * rng.randrange(100, 100_000)
                    for _ in range(transactions))
    return BankDataset(balances, ids, amounts)


def statistics_python(data: BankDataset) -> dict[str, int]:
    """Baseline: Python-цикли для суми балансу, обороту та статистики."""
    total = 0
    for balance in data.balances:
        total += balance
    incoming = outgoing = count_in = count_out = 0
    for amount in data.amounts:
        if amount >= 0:
            incoming += amount
```

```
        else:
            outgoing -= amount
            count_out += 1
    return {'total_balance': total, 'incoming': incoming, 'outgoing': outgoing,
            'turnover': incoming + outgoing, 'deposits': count_in, 'withdrawals': count_out}


def statistics_numpy(data: BankDataset) -> dict[str, int]:
    balances = np.asarray(data.balances, dtype=np.int64)
    amounts = np.asarray(data.amounts, dtype=np.int64)
    deposits = amounts >= 0
    incoming = int(amounts[deposits].sum())
    outgoing = int(-amounts[~deposits].sum())
    return {'total_balance': int(balances.sum()), 'incoming': incoming,
            'outgoing': outgoing, 'turnover': incoming + outgoing,
            'deposits': int(np.count_nonzero(deposits)),
            'withdrawals': int(np.count_nonzero(~deposits))}


def history_balance_python(initial_cents: int, movements: tuple[int, ...]) -> int:
    """Перерахунок історії з перевіркою заборони від'ємного балансу."""
    balance = initial_cents
    for movement in movements:
        if balance + movement < 0:
            raise ValueError('Недостатньо коштів')
        balance += movement
    return balance


def account_turnover_python(ids: tuple[int, ...], amounts: tuple[int, ...], account_id: int) -> int:
    return sum(abs(value) for idx, value in zip(ids, amounts) if idx == account_id)
```

```
class CachedAccountAnalytics:
    """Індексований підрахунок із очищенням кешу після зміни журналу."""
    def __init__(self, ids: tuple[int, ...], amounts: tuple[int, ...]):
        self.by_account: dict[int, list[int]] = defaultdict(list)
        for account_id, amount in zip(ids, amounts):
            self.by_account[account_id].append(amount)

    @lru_cache(maxsize=1024)
    def turnover(self, account_id: int) -> int:
        return sum(abs(amount) for amount in self.by_account.get(account_id, ()))

    def add_transaction(self, account_id: int, amount: int) -> None:
        self.by_account[account_id].append(amount)
        self.turnover.cache_clear()
```

3. Векторизація обчислень за допомогою NumPy

Для оптимізації числових операцій реалізовано функцію statistics_numpy().

На відміну від базової реалізації Python, у цій функції використовується бібліотека NumPy, яка забезпечує ефективну обробку числових масивів за допомогою векторизованих операцій.

Застосування NumPy дозволило скоротити час виконання банківської аналітики з 0,024907 с до 0,015849 с, що відповідає прискоренню приблизно в 1,57 раза.

Результати оптимізованих обчислень перевіряються на відповідність базовій реалізації.

```
def statistics_numpy(data: BankDataset) -> dict[str, int]:
    balances = np.asarray(data.balances, dtype=np.int64)
    amounts = np.asarray(data.amounts, dtype=np.int64)
    deposits = amounts >= 0
    incoming = int(amounts[deposits].sum())
    outgoing = int(-amounts[~deposits].sum())
    return {'total_balance': int(balances.sum()), 'incoming': incoming,
            'outgoing': outgoing, 'turnover': incoming + outgoing,
            'deposits': int(np.count_nonzero(deposits)),
            'withdrawals': int(np.count_nonzero(~deposits))}
```

4. Багатопотокова та багатопроцесна обробка

У файлі src/bank_system/parallel.py реалізовано декілька способів паралельного виконання обчислень.
Функція aggregate_threads() використовує ThreadPoolExecutor для розподілу задач між потоками. Функція aggregate_processes() використовує ProcessPoolExecutor для виконання незалежних обчислень в окремих процесах.
Також реалізовано приклади безпосереднього створення потоків та процесів за допомогою threading.Thread і multiprocessing.Process.
У ході експерименту було порівняно виконання з 1, 2, 4 та 8 workers. Отримані результати показали, що збільшення кількості потоків або процесів не завжди прискорює програму через GIL та додаткові витрати на організацію паралельного виконання.

```python
def summarize_chunk(values: tuple[int, ...]) -> tuple[int, int, int, int]:
    incoming = outgoing = deposits = withdrawals = 0
    for amount in values:
        if amount >= 0:
            incoming += amount
            deposits += 1
        else:
            outgoing -= amount
            withdrawals += 1
    return incoming, outgoing, deposits, withdrawals


def chunks(values: tuple[int, ...], pieces: int) -> list[tuple[int, ...]]:
    size = max(1, (len(values) + pieces - 1) // pieces)
    return [values[pos:pos + size] for pos in range(0, len(values), size)]


def aggregate_processes(values: tuple[int, ...], workers: int = 4) -> tuple[int, int, int, int]:
    with ProcessPoolExecutor(max_workers=workers) as pool:
        results = list(pool.map(summarize_chunk, chunks(values, workers * 4)))
    return tuple(sum(row[i] for row in results) for i in range(4))


def aggregate_threads(values: tuple[int, ...], workers: int = 4) -> tuple[int, int, int, int]:
    with ThreadPoolExecutor(max_workers=workers) as pool:
        results = list(pool.map(summarize_chunk, chunks(values, workers * 4)))
    return tuple(sum(row[i] for row in results) for i in range(4))


def _read_log(path: str) -> tuple[int, int]:
    with open(path, encoding='utf-8') as stream:
        lines = stream.readlines()
    return len(lines), sum(int(line.strip().split(',')[1]) for line in lines)
```

5. Синхронізація потоків та використання Lock

Для дослідження проблеми одночасного доступу до спільних даних було реалізовано демонстрацію race condition.
Декілька потоків виконують операції над спільним значенням. Без синхронізації частина оновлень втрачається, що призводить до неправильного результату.
Для усунення цієї проблеми використовується threading.Lock, який забезпечує взаємне виключення під час зміни спільного стану.

Під час експерименту без синхронізації було отримано значення 184 замість очікуваних 800. Після використання Lock результат становив 800, що підтвердило правильність синхронізованого виконання.

```python
class SafeBankAccount:
    def __init__(self, balance: int):
        if balance < 0: raise ValueError('Від'ємний баланс')
        self.balance = balance
        self.lock = Lock()

    def deposit(self, amount: int) -> None:
        if amount <= 0: raise ValueError('Сума має бути додатною')
        with self.lock:
            self.balance += amount

    def withdraw(self, amount: int) -> bool:
        if amount <= 0: raise ValueError('Сума має бути додатною')
        with self.lock:
            if self.balance < amount: return False
            self.balance -= amount
            return True
```

6. Оптимізація повторних обчислень за допомогою кешування

Для зменшення часу повторного виконання однакових обчислень реалізовано клас CachedAccountAnalytics.

Кешування дозволяє повторно використовувати попередньо розраховані результати статистики рахунків без необхідності здійснювати повторний перегляд усього набору транзакцій.

Після зміни даних кеш очищається, що забезпечує актуальність результатів (Cache Invalidation).

Під час експерименту повторне обчислення без кешу тривало 0,284205 с, а з використанням кешу — приблизно 0,000003 с. Це демонструє високу ефективність кешування для повторних запитів.

```python
class CachedAccountAnalytics:
    """Індексований підрахунок із очищенням кешу після зміни журналу."""
    def __init__(self, ids: tuple[int, ...], amounts: tuple[int, ...]):
        self.by_account: dict[int, list[int]] = defaultdict(list)
        for account_id, amount in zip(ids, amounts):
            self.by_account[account_id].append(amount)

    @lru_cache(maxsize=1024)
    def turnover(self, account_id: int) -> int:
        return sum(abs(amount) for amount in self.by_account.get(account_id, ()))

    def add_transaction(self, account_id: int, amount: int) -> None:
        self.by_account[account_id].append(amount)
        self.turnover.cache_clear()
```

7. Профілювання та автоматизація вимірювань

Для експериментального дослідження продуктивності було створено файл benchmarks/benchmark_lab9.py.
У програмі реалізовано вимірювання часу виконання за допомогою perf_counter і timeit. Для підвищення достовірності результатів кожне основне вимірювання виконується п'ять разів.
Для аналізу вузьких місць застосовано cProfile, який дозволяє визначити функції з найбільшим часом виконання.
За допомогою tracemalloc здійснюється аналіз використання пам'яті та визначення її пікового значення.
Результати вимірювань автоматично зберігаються у каталозі benchmarks/results/ у вигляді таблиць, текстових файлів та графіків. Це дозволяє порівняти реалізації та визначити ефективність застосованих методів оптимізації.

```python
def measure(function, repeats=5):
    times = []
    result = None
    for _ in range(repeats):
        start = perf_counter()
        result = function()
        times.append(perf_counter() - start)
    return statistics.median(times), result

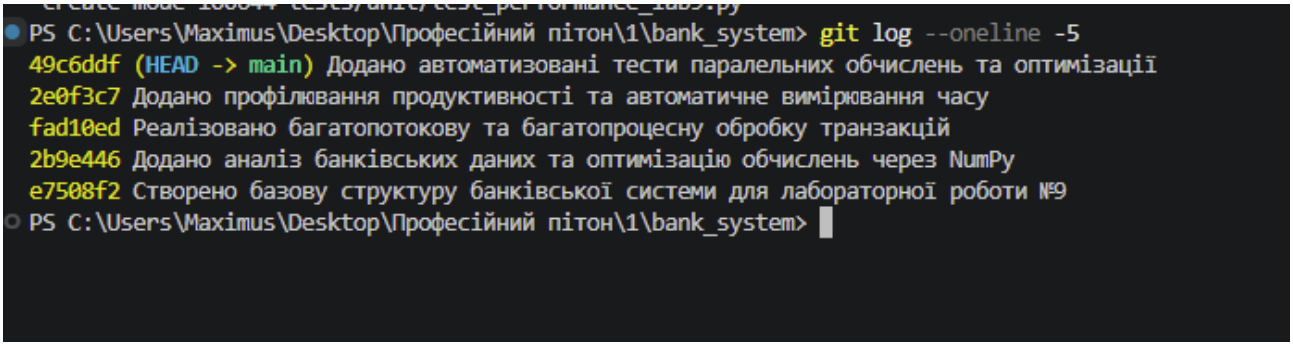

def measure_memory(function):
    tracemalloc.start()
    function()
    _, peak = tracemalloc.get_traced_memory()
    snapshot = tracemalloc.take_snapshot()
    tracemalloc.stop()
    return peak / 1048576, snapshot
```

4. Робота з Git

Для відстеження змін у 9 лабораторній роботі було виконано серію логічних комітів:

```
 create mode 100644 src/bank_system/value_objects.py
PS C:\Users\Maximus\Desktop\Професійний пітон\1\bank_system> git add src/bank_system/performance_analytics.py
warning: in the working copy of 'src/bank_system/performance_analytics.py', LF will be replaced by CRLF the next time Git touches it
PS C:\Users\Maximus\Desktop\Професійний пітон\1\bank_system> git commit -m "Додано аналіз банківських даних та оптимізацію обчислень через NumPy"
[main 2b9e446] Додано аналіз банківських даних та оптимізацію обчислень через NumPy
 1 file changed, 92 insertions(+)
 create mode 100644 src/bank_system/performance_analytics.py
PS C:\Users\Maximus\Desktop\Професійний пітон\1\bank_system> git add src/bank_system/parallel.py
warning: in the working copy of 'src/bank_system/parallel.py', LF will be replaced by CRLF the next time Git touches it
PS C:\Users\Maximus\Desktop\Професійний пітон\1\bank_system> git commit -m "Реалізовано багатопотокову та багатопроцесну обробку транзакцій"
[main fad10ed] Реалізовано багатопотокову та багатопроцесну обробку транзакцій
 1 file changed, 120 insertions(+)
 create mode 100644 src/bank_system/parallel.py
PS C:\Users\Maximus\Desktop\Професійний пітон\1\bank_system> git add benchmarks/benchmark_lab9.py
warning: in the working copy of 'benchmarks/benchmark_lab9.py', LF will be replaced by CRLF the next time Git touches it
PS C:\Users\Maximus\Desktop\Професійний пітон\1\bank_system> git commit -m "Додано профілювання продуктивності та автоматичне вимірювання часу"
[main 2e0f3c7] Додано профілювання продуктивності та автоматичне вимірювання часу
 1 file changed, 167 insertions(+)
 create mode 100644 benchmarks/benchmark_lab9.py
PS C:\Users\Maximus\Desktop\Професійний пітон\1\bank_system> git add tests/unit/test_performance_lab9.py
warning: in the working copy of 'tests/unit/test_performance_lab9.py', LF will be replaced by CRLF the next time Git touches it
PS C:\Users\Maximus\Desktop\Професійний пітон\1\bank_system> git commit -m "Додано автоматизовані тести паралельних обчислень та оптимізації"
[main 49c6ddf] Додано автоматизовані тести паралельних обчислень та оптимізації
 1 file changed, 52 insertions(+)
 create mode 100644 tests/unit/test_performance_lab9.py
PS C:\Users\Maximus\Desktop\Професійний пітон\1\bank_system>
```

Перевірка історії комітів (git log --oneline):



Індивідуальне завдання (Варіант 4)

Тема: Профілювання та оптимізація продуктивності системи банківських рахунків із використанням паралельних обчислень у Python.

Відповідно до індивідуального варіанта №4 було виконано розширення існуючої системи банківських рахунків шляхом реалізації алгоритмів обробки великих обсягів фінансових даних, багатопотокових і багатопроцесних обчислень, векторизації NumPy та кешування.

Основною метою індивідуального завдання стало експериментальне дослідження продуктивності різних способів обробки банківських операцій, визначення вузьких місць програми (bottlenecks) та вибір найефективнішого методу оптимізації.

Для проведення експериментів було сформовано набір із **100 000 банківських рахунків та 500 000 транзакцій**.

Архітектура рішення та реалізовані компоненти

1. Генерація тестових даних та базова реалізація (performance_analytics.py)

Реалізовано генерацію великого набору банківських рахунків і транзакцій. Для обробки даних створено послідовний алгоритм Python (Baseline), який виконує обчислення фінансової статистики та використовується як основа для порівняння продуктивності.

2. Оптимізація обчислень за допомогою NumPy

Реалізовано векторизовану обробку числових даних із використанням бібліотеки NumPy. Завдяки заміні частини послідовних обчислень операціями над масивами вдалося зменшити час виконання з 0,024907 с до 0,015849 с, що відповідає прискоренню в 1,57 раза.

3. Багатопотокова та багатопроцесна обробка (parallel.py)

Реалізовано виконання операцій за допомогою ThreadPoolExecutor та ProcessPoolExecutor. Проведено дослідження впливу кількості workers (1, 2, 4, 8) на продуктивність програми.

Результати показали, що використання додаткових потоків і процесів не завжди забезпечує прискорення, особливо для відносно коротких обчислювальних задач.

4. Синхронізація потоків та запобігання Race Condition

Реалізовано моделювання одночасного оновлення спільного стану декількома потоками.

Під час експерименту без синхронізації отримано неправильний результат — 184 замість 800. Використання threading.Lock дозволило забезпечити правильне виконання всіх операцій і отримати очікуваний результат 800.

5. Кешування результатів фінансової аналітики

Для оптимізації повторних звернень реалізовано механізм кешування результатів обчислень.

Повторне звернення без кешування тривало 0,284205 с, тоді як із кешуванням — приблизно 0,000003 с.

Також реалізовано механізм Cache Invalidation, який забезпечує актуальність результатів після зміни вихідних даних.

6. Профілювання та аналіз продуктивності (benchmark_lab9.py)

Для вимірювання продуктивності застосовано perf_counter, timeit, cProfile та tracemalloc.

Реалізовано автоматизоване проведення експериментів із п'ятьма повторами, розрахунок коефіцієнтів прискорення та збереження результатів у каталог benchmarks/results/.

7. Автоматизоване тестування (pytest)

Для перевірки правильності роботи алгоритмів застосовано фреймворк pytest.

Після виконання команди python -m pytest -q усі 15 тестів успішно пройшли перевірку (15 passed). Це підтверджує коректність перевірених функціональних компонентів банківської системи.

Результат виконання індивідуального завдання

У результаті виконання індивідуального завдання було досліджено продуктивність банківської системи на великих обсягах даних, реалізовано декілька способів паралельної обробки та застосовано методи оптимізації.

Експериментально встановлено, що для реалізованої числової аналітики найбільш ефективним методом виявилася векторизація NumPy, яка забезпечила прискорення в 1,57 раза.

Використання Lock дозволило усунути помилки конкурентного оновлення спільного стану, а кешування значно скоротило час повторних обчислень.

Усі 15 автоматизованих тестів завершилися успішно, що підтверджує правильність реалізованих та перевірених програмних компонентів.

1. Результати автоматизованого тестування (pytest)

Для перевірки коректності роботи банківської системи після реалізації алгоритмів оптимізації було проведено автоматизоване тестування за допомогою фреймворку pytest.

Тестування охоплювало перевірку функціональності банківської системи, правильності фінансових операцій, паралельної обробки даних, кешування та синхронізації потоків.

Для запуску тестів у терміналі VS Code було виконано команду:

python -m pytest -q

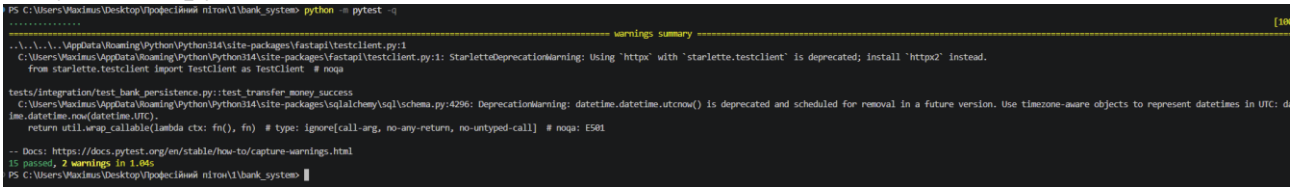
За результатами перевірки всі 15 тестів пройшли успішно:

15 passed, 2 warnings in 1.25s

Два попередження стосувалися використання застарілих механізмів сторонніх бібліотек Starlette та SQLAlchemy і не вплинули на результати тестування.

Отже, проведене тестування підтвердило правильність роботи перевірених компонентів банківської системи.

Результати pytest



2. Результати вимірювання продуктивності

Для дослідження продуктивності банківської системи було використано тестовий набір із 100 000 рахунків і 500 000 транзакцій.

Вимірювання виконувалися за допомогою скрипту benchmarks/benchmark_lab9.py. Для основних алгоритмів використовувалося п'ять повторів із подальшим визначенням середнього часу виконання.

Для порівняння використовувалися послідовна реалізація Python, векторизована реалізація NumPy, ThreadPoolExecutor та ProcessPoolExecutor.

Команда запуску:

python benchmarks/benchmark_lab9.py

За отриманими результатами час виконання базової реалізації становив 0,024907 с, а векторизованої реалізації NumPy — 0,015849 с.

Таким чином, використання NumPy забезпечило прискорення приблизно в 1,57 раза.

Окремо було досліджено паралельний розрахунок обороту транзакцій. Послідовна реалізація цієї операції виконувалася за 0,022059 с.

Результати експерименту наведено в таблицях нижче.

```
PS C:\Users\Maximus\Desktop\Професійний пітон\1\bank_system> python benchmarks/benchmark_lab9.py
Рахунків: 100000; транзакцій: 500000; повторів: 5
Python baseline                0.024907 s | x1.00 | peak=0.00 MB
NumPy vectorization            0.015849 s | x1.57 | peak=7.44 MB
Python turnover only           0.022059 s | x1.00 | peak=0.00 MB
ThreadPool CPU                 0.026982 s | x0.82
ProcessPool CPU                0.364139 s | x0.06
ThreadPool CPU                 0.027820 s | x0.79
ProcessPool CPU                0.316257 s | x0.07
ThreadPool CPU                 0.029914 s | x0.74
ProcessPool CPU                0.347647 s | x0.06
ThreadPool CPU                 0.031727 s | x0.70
ProcessPool CPU                0.466936 s | x0.05
Sequential CSV reading         0.021533 s | x1.00
ThreadPool CSV reading         0.022955 s | x0.94
Uncached repeated lookup       0.284205 s | x1.00
Cached repeated lookup         0.000003 s | x83588.98
Cache invalidation: OK; info: CacheInfo(hits=0, misses=1, maxsize=1024, currsize=1)
Race condition unsafe/safe: (184, 800)
timeit, 100 warm cached reads: 1.0600022505968809e-05
Результати збережено: C:\Users\Maximus\Desktop\Професійний пітон\1\bank_system\benchmarks\results
```

Таблиця виконання вимог

№	Вимога лабораторної роботи №9	Статус	Реалізація у проєкті (Варіант 4)
1	Продовження попереднього проєкту	Виконано	Використано існуючий проєкт `bank_system` із попередніх лабораторних робіт.
2	Створення великого набору даних	Виконано	Згенеровано 100 000 банківських рахунків і 500 000 транзакцій.
3	Реалізація Baseline	Виконано	Створено базову послідовну реалізацію банківської аналітики на Python.
4	Визначення CPU-bound та I/O-bound задач	Виконано	Досліджено обчислення статистики транзакцій і читання CSV-файлів.
5	Використання perf_counter	Виконано	Реалізовано вимірювання часу виконання операцій.
6	Використання timeit	Виконано	Проведено окреме вимірювання швидкодії повторних звернень до кешу.
7	Профілювання через cProfile	Виконано	Реалізовано профілювання для визначення найбільш витратних функцій.
8	Аналіз пам'яті через tracemalloc	Виконано	Виміряно пікове використання пам'яті, зокрема 7,44 МБ для NumPy.
9	Виявлення Bottleneck	Виконано	Досліджено витрати часу на обробку великих масивів фінансових даних.
10	Використання threading	Виконано	Реалізовано приклади створення та виконання потоків.
11	Використання ThreadPoolExecutor	Виконано	Реалізовано багатопотокову обробку транзакцій і CSV-файлів.
12	Використання multiprocessing	Виконано	Реалізовано виконання незалежних обчислень в окремих процесах.
13	Використання ProcessPoolExecutor	Виконано	Досліджено багатопроцесне обчислення фінансового обороту.
14	Синхронізація через Lock	Виконано	Продемонстровано Race Condition та виправлення результату з 184 до 800.
15	Оптимізація через NumPy	Виконано	Реалізовано векторизовані обчислення з прискоренням 1,57×.
16	Реалізація Caching	Виконано	Створено кешування повторних обчислень банківської статистики.
17	Cache Invalidation	Виконано	Перевірено очищення кешу після зміни даних — `Cache invalidation: OK`.
18	Порівняння щонайменше трьох реалізацій	Виконано	Порівняно Python, NumPy, ThreadPoolExecutor та ProcessPoolExecutor.
19	Багаторазове виконання Benchmark	Виконано	Основні вимірювання проводилися з п'ятьма повторами.
20	Обчислення Speedup	Виконано	Розраховано коефіцієнти прискорення для досліджуваних реалізацій.
21	Дослідження кількості Workers	Виконано	Проведено експерименти з 1, 2, 4 та 8 потоками й процесами.
22	Експеримент із I/O-bound операціями	Виконано	Порівняно послідовне та багатопотокове читання CSV-файлів.
23	Перевірка правильності результатів	Виконано	Перевірено відповідність результатів обчислень та синхронізації.

24	Формування таблиць результатів	Виконано	Результати Benchmark збережено у `benchmarks/results/`.
25	Побудова графіків продуктивності	Виконано	Створено графіки часу виконання, прискорення та порівняння NumPy.
26	Автоматизоване тестування	Виконано	Запущено pytest: 15 тестів успішно пройшли перевірку.
27	Аналіз отриманих результатів	Виконано	Встановлено перевагу NumPy для досліджуваної числової аналітики.
28	Оновлення структури проєкту	Виконано	Додано модулі аналітики, паралельної обробки, тестування та Benchmark.
29	Використання Git	Виконано	Підготовлено серію комітів українською мовою для лабораторної №9. Відправлення до GitHub потребує перевірки.
30	Формування звіту	Виконано	Підготовлено опис реалізації, експериментальні результати, таблиці та графіки.

Контрольні запитання

1. Що таке concurrency?

Concurrency (конкурентність) — це організація виконання декількох задач, за якої програма може перемикатися між ними, створюючи можливість їх одночасного просування.

2. Що таке parallelism?

Parallelism (паралелізм) — це фізично одночасне виконання декількох задач на різних ядрах процесора.

3. Чим concurrency відрізняється від parallelism?

Concurrency дозволяє перемикатися між задачами, а parallelism забезпечує їх фактичне одночасне виконання.

4. Що таке process?

Process (процес) — це окремий екземпляр програми, який має власну пам'ять та системні ресурси.

5. Що таке thread?

Thread (потік) — це одиниця виконання всередині процесу, яка використовує спільну пам'ять з іншими потоками цього процесу.

6. Чим process відрізняється від thread?

Процеси мають окремі адресні простори, а потоки одного процесу використовують спільну пам'ять. Створення потоків зазвичай потребує менше ресурсів.

7. Що таке GIL?

GIL (Global Interpreter Lock) — це глобальне блокування інтерпретатора Python, яке обмежує одночасне виконання Python-байткоду потоками одного процесу у звичайному CPython.

8. Яку роль GIL відіграє у CPython?

GIL забезпечує синхронізацію виконання Python-байткоду та захищає внутрішні структури інтерпретатора, але може обмежувати продуктивність багатопотокових CPU-bound задач.

9. Що таке CPU-bound задача?

CPU-bound — це задача, швидкість виконання якої переважно залежить від продуктивності процесора.

10. Наведіть приклад CPU-bound задачі.

Обчислення загального обороту та статистичних показників для 500 000 банківських транзакцій.

11. Що таке I/O-bound задача?

I/O-bound — це задача, яка значну частину часу витрачає на очікування операцій введення-виведення.

12. Наведіть приклад I/O-bound задачі.

Читання CSV-файлів із банківськими транзакціями або отримання даних із бази даних.

13. Коли доцільно застосовувати threading?

Threading доцільно використовувати для I/O-bound задач, наприклад читання файлів, мережевих запитів та роботи із зовнішніми сервісами.

14. Коли доцільно застосовувати multiprocessing?

Multiprocessing доцільно використовувати для складних CPU-bound обчислень, які можна розподілити між декількома процесорними ядрами.

15. Для чого використовується Thread?

Клас Thread із модуля threading використовується для створення та керування окремими потоками виконання.

16. Для чого використовується start()?

Метод start() запускає виконання потоку або процесу.

17. Для чого використовується join()?

Метод join() очікує завершення відповідного потоку або процесу.

18. Що таке race condition?

Race condition — це ситуація, коли результат роботи програми залежить від непередбачуваного порядку виконання потоків або процесів.

19. Що таке critical section?

Critical section (критична секція) — це частина програми, у якій здійснюється доступ до спільних даних і яка потребує синхронізації.

20. Для чого використовується Lock?

Lock використовується для блокування одночасного доступу декількох потоків до критичної секції коду.

21. Що таке synchronization?

Synchronization (синхронізація) — це узгодження виконання потоків або процесів для забезпечення правильного доступу до спільних ресурсів.

22. Чи завжди synchronization покращує performance?

Ні. Синхронізація забезпечує правильність виконання, але може знижувати продуктивність через очікування блокувань.

23. Що таке ThreadPoolExecutor?
ThreadPoolExecutor — це клас із модуля concurrent.futures, який дозволяє виконувати задачі за допомогою пулу потоків.
24. Коли його доцільно використовувати?
Його доцільно використовувати для одночасної обробки незалежних I/O-bound задач, наприклад читання декількох файлів.
25. Що таке ProcessPoolExecutor?
ProcessPoolExecutor — це клас, який дозволяє паралельно виконувати задачі в окремих процесах.
26. Коли його доцільно використовувати?
Його доцільно використовувати для ресурсоємних CPU-bound обчислень, які можна розподілити між декількома процесами.
27. Для чого використовується executor.map()?
Метод executor.map() застосовує задану функцію до елементів послідовності, розподіляючи їх обробку між потоками або процесами.
28. Що таке worker?
Worker — це потік або процес, який виконує окремі задачі, отримані від керувального механізму.
29. Чи завжди більша кількість workers дає більшу швидкість?
Ні. Надмірна кількість workers може збільшувати накладні витрати, використання пам'яті та час виконання.
30. Що таке multiprocessing overhead?
Multiprocessing overhead — це додаткові витрати часу та ресурсів на створення процесів, обмін даними, серіалізацію й керування виконанням.
31. Що таке serialization?
Serialization (серіалізація) — це перетворення об'єктів у формат, придатний для збереження або передавання між процесами.
32. Чому multiprocessing може бути повільним для маленьких задач?
Тому що витрати на створення процесів і передавання даних можуть перевищувати час виконання самих обчислень.
33. Для чого використовується timeit?
Модуль timeit використовується для точного вимірювання часу виконання невеликих фрагментів Python-коду.
34. Для чого використовується perf_counter()?
Функція perf_counter() використовується для вимірювання тривалості виконання операцій із високою роздільною здатністю таймера.
35. Навіщо виконувати декілька benchmark repeats?
Повторні вимірювання дозволяють зменшити вплив випадкових коливань продуктивності та визначити більш достовірний середній час виконання.
36. Що таке profiling?
Profiling (профілювання) — це аналіз роботи програми для визначення функцій, які використовують найбільше часу або пам'яті.
37. Для чого використовується cProfile?

cProfile використовується для вимірювання кількості викликів функцій і часу їх виконання з метою пошуку вузьких місць програми.

38. Що показує ncalls?

ncalls показує кількість викликів функції під час виконання програми.

39. Що показує tottime?

tottime показує час, витрачений безпосередньо всередині функції, без урахування часу її вкладених викликів.

40. Що показує cumtime?

cumtime показує загальний час виконання функції разом із часом усіх вкладених викликів.

41. Що таке bottleneck?

Bottleneck (вузьке місце) — це частина програми, яка найбільше обмежує її загальну продуктивність.

42. Як знайти bottleneck?

Необхідно виконати профілювання програми за допомогою cProfile, проаналізувати час виконання функцій і визначити найбільш витратні операції.

43. Для чого використовується tracemalloc?

tracemalloc використовується для відстеження виділення пам'яті Python-програмою та пошуку джерел її використання.

44. Що таке peak memory?

Peak memory — це максимальний обсяг пам'яті, зафіксований під час виконання досліджуваної операції. Для tracemalloc це піковий обсяг відстежуваних виділень пам'яті.

45. Що таке NumPy?

NumPy — це бібліотека Python для ефективної роботи з багатовимірними числовими масивами та математичними обчисленнями.

46. Що таке vectorization?

Vectorization (векторизація) — це виконання операцій над цілими масивами даних без необхідності явного перебору кожного елемента Python-циклом.

47. Чому NumPy часто швидший за Python loops?

Тому що NumPy виконує багато числових операцій за допомогою оптимізованого низькорівневого коду, зменшуючи накладні витрати інтерпретатора Python.

48. Що таке caching?

Caching (кешування) — це збереження результатів попередніх обчислень для їх швидкого повторного використання.

49. Для чого використовується @cache?

Декоратор @cache із модуля functools автоматично зберігає результати викликів функції для повторного використання з тими самими аргументами без обмеження кількості записів.

50. Для чого використовується @lru_cache?

Декоратор @lru_cache кешує результати функції та може обмежувати розмір кешу, видаляючи записи, які найдовше не використовувалися.

51. Що таке cache hit?
Cache hit — це ситуація, коли потрібний результат уже міститься в кеші та повертається без повторного обчислення.
52. Що таке cache miss?
Cache miss — це ситуація, коли необхідного результату немає в кеші, тому функція повинна виконати обчислення.
53. Що таке cache invalidation?
Cache invalidation — це очищення або оновлення кешованих даних після зміни вихідної інформації.
54. Коли кешування може бути небезпечним?
Коли вихідні дані змінюються, а кеш не оновлюється. У такому випадку програма може повертати застарілі або неправильні результати.
55. Як обчислюється speedup?
Коефіцієнт прискорення обчислюється за формулою:
Speedup = T_baseline / T_optimized,
де T_baseline — час базової реалізації, а T_optimized — час оптимізованої.
56. Що означає speedup менше 1?
Speedup менше 1 означає, що оптимізована або паралельна реалізація працює повільніше за базову.
57. Чому threaded CPU-bound version може бути повільнішою?
Через обмеження GIL у звичайному CPython та додаткові витрати на створення потоків, перемикання між ними й синхронізацію.
58. Чому ProcessPoolExecutor може прискорити CPU-bound код?
Тому що кожен процес має власний інтерпретатор і GIL, що дозволяє виконувати незалежні обчислення одночасно на декількох ядрах процесора.
59. Чому необхідно перевіряти правильність optimized implementation?
Тому що оптимізація не повинна змінювати логіку програми або призводити до неправильних результатів обчислень.
60. Чому оптимізацію потрібно починати з profiling?
Тому що профілювання дозволяє визначити реальні вузькі місця програми та спрямувати оптимізацію на ті операції, які найбільше впливають на продуктивність.

Висновок
Під час виконання лабораторної роботи №9 було досліджено основні підходи до паралельного та конкурентного виконання програм у Python, а також методи профілювання й оптимізації продуктивності. На основі раніше розробленої системи банківських рахунків було реалізовано обробку великого набору даних, що містив 100 000 рахунків і 500 000 транзакцій.
Було проведено порівняння послідовної обробки Python, багатопотокових обчислень через ThreadPoolExecutor, багатопроцесної обробки через ProcessPoolExecutor та векторизованих операцій NumPy. Найкращий результат для основної числової аналітики продемонструвала бібліотека NumPy, яка

скоротила час виконання з 0,024907 с до 0,015849 с, забезпечивши прискорення в 1,57 раза.

У ході експериментів встановлено, що використання потоків і процесів не завжди підвищує продуктивність через вплив GIL та додаткові витрати на організацію паралельного виконання. Також було реалізовано кешування повторних обчислень, що дозволило суттєво скоротити час отримання результатів. За допомогою механізму Lock вдалося усунути проблему Race Condition та забезпечити правильність конкурентного оновлення спільних даних.

Для аналізу продуктивності застосовано perf_counter, timeit, cProfile та tracemalloc. Проведено багаторазові вимірювання часу виконання, досліджено використання пам'яті та побудовано графіки порівняння алгоритмів. Правильність роботи програмних компонентів перевірено за допомогою pytest — усі 15 тестів успішно пройшли перевірку.

У результаті лабораторної роботи було отримано практичні навички використання багатопотоковості, багатопроцесності, синхронізації, кешування та векторизації. Експериментально підтверджено, що вибір методу оптимізації повинен ґрунтуватися на результатах профілювання та вимірювання продуктивності конкретної задачі.